

REVISÃO DE NODE



1. FUNDAMENTOS DO NODE.JS

O que o runtime faz, como ele executa JavaScript fora do navegador e por que sua arquitetura assíncrona importa.

Node.js em uma frase

Node.js é um runtime JavaScript construído sobre o motor V8. Ele permite executar JavaScript no servidor, acessar arquivos, rede, processos e construir aplicações como APIs, CLIs, workers e serviços.

A grande vantagem não é apenas “usar JavaScript no backend”, mas trabalhar com I/O não bloqueante: enquanto uma operação de rede ou disco aguarda resposta, o processo pode continuar tratando outras tarefas.

- Runtime ≠ framework.
- V8 executa JavaScript; Node adiciona APIs de sistema.
- Node é muito forte em aplicações orientadas a I/O e grande concorrência.

Event Loop e concorrência

O JavaScript roda em uma thread principal, mas o Node delega várias operações ao sistema operacional e ao libuv. Quando elas terminam, callbacks e promises voltam para filas que serão processadas pelo event loop.

- Evite bloquear a thread principal com cálculos pesados.
- CPU-bound intenso pode exigir Worker Threads, filas ou serviços separados.
- Promises e async/await não tornam código CPU-bound paralelo.

Exemplo básico

```
const fs = require('fs/promises');

async function main() {
  const data = await fs.readFile('data.txt', 'utf8');
  console.log(data);
}

main();
```

2. MÓDULOS, NPM E ESTRUTURA DE PROJETO

Como o código é organizado, como dependências entram no projeto e como evitar um backend que vira um único arquivo gigante.

CommonJS x ES Modules

Node suporta dois sistemas principais de módulos. CommonJS usa `require/module.exports`; ES Modules usa `import/export`. Projetos novos podem usar ESM com `"type": "module"` no `package.json`.

- Não misture estilos sem necessidade.
- Consistência de importações reduz erros e melhora tooling.

package.json e dependências

O `package.json` descreve scripts, versão do projeto e dependências. Em produção, `dependencies` são necessárias para executar a aplicação; `devDependencies` são ferramentas de desenvolvimento, testes, lint e build.

- `npm install` instala dependências.
- `npm run <script>` executa scripts declarados.
- `package-lock.json` deve ser versionado para builds reproduzíveis.

Estrutura saudável

Uma API costuma ficar mais clara quando separa responsabilidades: rotas recebem a requisição, controllers coordenam, services concentram regras de negócio e repositories/data access lidam com persistência.

- Evite SQL espalhado em componentes sem padrão.
- Evite colocar regra de negócio complexa diretamente nas rotas.

Estrutura típica

```
src/  
  routes/  
  controllers/  
  services/  
  repositories/  
  middlewares/  
  config/  
  server.js
```

3. ASSÍNCRONO, PROMISES E TRATAMENTO DE ERROS

O coração do backend Node: operações assíncronas previsíveis, erros tratados e controle de fluxo legível.

Promises e async/await

async/await é uma sintaxe sobre Promises. Ela torna sequências assíncronas mais legíveis, mas cada await ainda pode falhar. Por isso, operações críticas devem ter tratamento de erro e caminhos claros de recuperação.

- Use Promise.all quando tarefas independentes podem ocorrer em paralelo.
- Evite await em sequência quando as operações não dependem umas das outras.
- Não esqueça de retornar ou propagar erros.

Tratamento centralizado

Em APIs Express, vale centralizar erros em middleware. Controllers podem lançar ou encaminhar erros, e um único ponto transforma exceções em respostas HTTP consistentes.

- 400 para entrada inválida.
- 401 para não autenticado.
- 403 para autenticado sem permissão.
- 404 para recurso ausente.
- 500 para falhas inesperadas.

Erros operacionais x bugs

Nem todo erro deve derrubar o processo. Entrada inválida, recurso inexistente e conflito são erros esperados. Já exceções inesperadas exigem log detalhado e observabilidade.

Paralelismo com Promise.all

```
const [user, orders] = await Promise.all([
  getUser(userId),
  getOrders(userId)
]);
```

4. EXPRESS, APIS REST E MIDDLEWARES

Como transformar regras de negócio em endpoints previsíveis, seguros e fáceis de consumir pelo frontend.

Ciclo de uma requisição

Uma requisição chega à rota, passa por middlewares, chega ao controller e então pode chamar services e camada de dados. A resposta deve sair com status HTTP e payload coerentes.

- Rotas definem URL e método.
- Middlewares tratam preocupações transversais.
- Services devem concentrar regras importantes.
- Controllers não devem virar “services disfarçados”.

REST e semântica HTTP

GET lê, POST cria, PUT/PATCH atualizam e DELETE remove ou aciona a política de exclusão do domínio. O endpoint deve representar recursos e ações com clareza.

- GET /api/courses
- GET /api/courses/:id
- POST /api/courses
- PATCH /api/courses/:id

Middlewares úteis

Autenticação, autorização, validação, rate limiting, logging e tratamento de erros são bons candidatos a middleware porque aparecem em muitas rotas.

Exemplo de rota

```
router.get('/api/courses/:id', auth, async (req, res, next) => {
  try {
    const course = await courseService.getById(req.params.id);
    res.json(course);
  } catch (err) {
    next(err);
  }
});
```

5. BANCO DE DADOS, AUTENTICAÇÃO E SEGURANÇA

Persistência confiável e segurança não são detalhes do backend: são parte da regra de negócio.

Banco relacional

Com MySQL ou PostgreSQL, relações devem ser expressas com chaves estrangeiras, índices e constraints. O banco ajuda a impedir estados impossíveis, mas a aplicação ainda precisa validar regras de domínio.

- Use transações para mudanças que precisam acontecer juntas.
- Indexe colunas usadas em joins e filtros frequentes.
- Não use IDs enviados pelo cliente como prova de autorização.

Autenticação x autorização

Autenticação responde “quem é você?”. Autorização responde “o que você pode fazer aqui?”. Um usuário autenticado não deve automaticamente ter acesso a qualquer recurso.

- JWT pode representar a sessão/identidade.
- Refresh tokens exigem estratégia de revogação e armazenamento seguro.
- Cookies HTTP-only reduzem exposição de tokens ao JavaScript do navegador.

Boas práticas de segurança

Nunca armazene senha em texto puro. Use hashing adequado, valide entradas, limite informações de erro expostas ao cliente e mantenha segredos em variáveis de ambiente.

- bcrypt/argon2 para senhas.
- Prepared statements ou query builders para evitar SQL injection.
- CORS configurado explicitamente em produção.
- Rate limiting para endpoints sensíveis.

Transação conceitual

```
BEGIN;  
  INSERT INTO orders (...);  
  UPDATE inventory SET stock = stock - 1 WHERE id = ?;  
COMMIT;  
-- ROLLBACK se qualquer etapa falhar
```


6. TESTES, DEBUGGING, PERFORMANCE E DEPLOY

A aplicação só está pronta quando você consegue observar, testar e reproduzir seu comportamento fora do ambiente local.

Testes

Testes unitários validam pequenas regras isoladas; testes de integração verificam módulos trabalhando juntos; testes end-to-end percorrem fluxos completos como um usuário real.

- Teste regras de negócio críticas.
- Teste autenticação e autorização.
- Teste cenários de erro, não apenas o caminho feliz.

Debugging e observabilidade

Logs devem responder o que aconteceu, em qual rota, para qual requisição e qual erro ocorreu. Em produção, logs estruturados são mais úteis do que `console.log` solto.

- Inclua request IDs quando possível.
- Nunca logue senhas, tokens ou dados sensíveis.
- Monitore latência, erros e consumo de recursos.

Deploy e operação

Em produção, variáveis de ambiente, conexão com banco, migrations, health checks e estratégia de restart precisam estar definidos. Docker ajuda a tornar o ambiente reproduzível.

- Health check: `GET /api/health`.
- Use process manager/container orchestration.
- Garanta que migrations e seeds sejam controlados.
- CI/CD reduz deploy manual e inconsistências.

Checklist final

Uma boa revisão de Node combina sintaxe e arquitetura: entender event loop, módulos, `async/await`, APIs, banco, autenticação, testes e deploy é o que transforma código funcional em serviço confiável.

Scripts úteis

```
npm run dev
npm test
npm run lint
npm run migrate
npm start
```